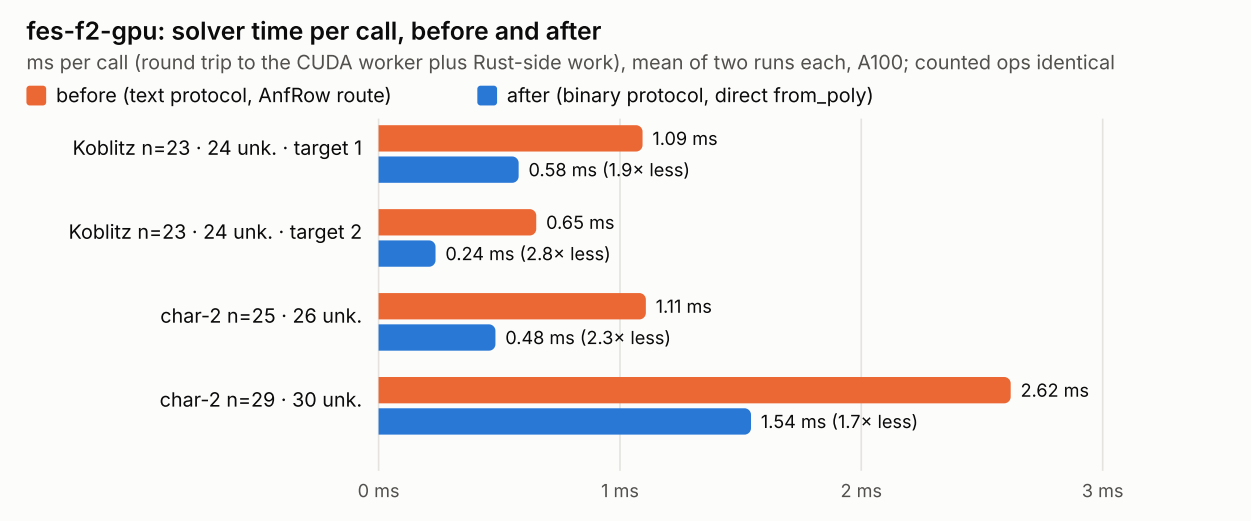
fes-f2-gpu: per-call overhead

User request (2026-11-13, “Yes onwards”): cut the per-call overhead of `fes-f2-gpu`, which `research/ic_gpu_e2e_20261113` measured at 1.0–3.6 ms per call against 0.1–1.3 ms of kernel time.

Standing. `[pcc:unverified]`: one pod per A/B, measured by the session that wrote the code, no validator. Counted ops, calls and hit rate are identical in every arm, so counted S does not move; the wall columns are a practicality note (`AGENTS.md` §6). Engineering class, no scoreboard row.

Two sessions worked on this at the same time. While this branch was open, another session landed `fes_pack_direct` on main (`research/ic_gpu_call_overhead_20261113`): the GPU engine packs the worker’s words straight from the polynomials, preparation 0.22 → 0.008 ms per call on a Modal A100. That is the larger part of the Rust-side cut measured in A/B 1 below, and in the merged tree it is the GPU engine’s preparation path. This branch’s own contribution on top of main is the binary protocol and single-copy worker I/O, measured in A/B 2, plus a quicker `QuadraticForm::from_poly` [pcc:unverified] for the CPU `fes-f2` and the form-path fallbacks.



A/B 1: solver time per call, abecd26901 against this branch, [pcc:unverified]

Where the overhead was

From the first end-to-end run’s JSON, about half of each call was outside the worker round trip, Rust-side: 0.44–0.66 ms per call at 24–26 unknowns, 1.75 ms at 30. A local probe on degree-29 descent systems (30 unknowns, 29 equations, 3542 terms; ignored test `fes_gpu_rust_side_overhead_probe`) put the engine body around the search at 0.569 ms, of which `AnfRow::from_poly` was 0.421 ms: `Mask` is `u128`, and every term scanned 128 bit positions and allocated a `Vec`. The other session’s worker timers left about 0.2 ms of the round trip unlocated, attributed to pipe and stdio latency.

Changes (commit `16e7903d26`)

- `QuadraticForm::from_poly` reads the term masks directly (popcount 0/1/2 toggles the constant, linear or quadratic coefficient; degree above 2 or a variable `>= n` gives `None`, as before). Test `from_poly_equals_the_anf_row_route`: equal to the `AnfRow` route on 3000 random polynomials,

rejections included; main's `direct_pack_equals_the_form_pack_and_its_charges` also passes on the merged tree. Probe: `from_poly` 0.461 → 0.041 ms, form-path engine body 0.569 → 0.062 ms.
- `AnfRow::from_poly` walks the set bits only (same lists, already sorted).
- `--serve-bin` (`fes_io.hpp`, host and CUDA workers): little-endian request and reply, no text. The CUDA worker uploads `[lin | quad | qsym]` from one pinned buffer in one copy, and the count comes back with the first 64 solutions in one copy and one synchronisation. `ServeWorker` speaks it; the serve test (40 systems against the host worker) covers it. On both pods the CUDA worker's replies equalled the host worker's on 30 systems (459 050 solutions).

Runs

Both on A100 80GB PCIe pods, CUDA 12.8, ABBA per sweep, same instances and seeds as `research/ic_gpu_e2e_20261113` (no mitm row). Counted ops, calls and hit rate are identical per repeat in every run of every arm; every row recovered the planted logarithm. S is the counted ops priced at the curve's calibration: pinned for the Koblitz curve (identical everywhere), measured once per `ic bench` run for the char-2 curves (about ±0.5% between runs of the same arm, any engine).

A/B 1: `abecd26901` (text serve, `AnfRow` route) against this branch before merging main

Pod `gvor4yci0yby96` (host `ruvct7owvq4l`), 04:37Z–04:57Z, about $0.70; `raw/out_ab.tgz`, `RESULTS.md`.

instance (unknowns)	target	solver ms/call	Rust side ms/call	whole solve s
Koblitz n=23 (24)	1	1.094 → 0.580 (1.89×)	0.49 → 0.10	1.07 → 0.64 (1.68×)
	2	0.653 → 0.236 (2.77×)	0.41 → 0.11	0.78 → 0.39 (1.97×)
char-2 n=25 (26)	1	1.107 → 0.484 (2.29×)	0.60 → 0.13	3.04 → 1.61 (1.89×)
char-2 n=29 (30)	1	2.619 → 1.543 (1.70×)	1.04 → 0.18	27.35 → 17.45 (1.57×)

Largest max/min between two runs of the same arm and target: 1.20 (n=25), 1.15 (n=23), 1.004 (n=29) [pcc:unverified]. CPU `fes-f2` at n=23: 13.65 → 13.44 ms per call (1.5%, same-arm spread 0.5%).

A/B 2: main `3e432229fc` (with `fes_pack_direct`, text serve) against the merged branch

Isolates `--serve-bin` and the single-copy I/O on top of main. Pod `9xe4bzapq1shkm` (host `pby1uyvbzoij`), 05:28Z–05:48Z, about $0.70; `raw/out_ab2.tgz`, `RESULTS2.md`.

instance (unknowns)	target	solver ms/call main → merged	whole solve s	same-arm max/min
Koblitz n=23 (24)	1	0.527 → 0.539 (no change)	0.53 → 0.53	1.31
	2	0.204 → 0.129 (1.59×)	0.28 → 0.20 (1.43×)	
char-2 n=25 (26)	1	0.423 → 0.367 (1.15×)	1.25 → 1.11 (1.12×)	1.09

instance (unknowns)	target	solver ms/call main → merged	whole solve s	same-arm max/min
char-2 n=29 (30)	1	1.547 → 1.443 (1.07×)	16.40 → 15.17 (1.08×)	1.000

Reading [pcc:unverified]: the binary protocol takes about 0.06–0.10 ms off each call at 24–30 unknowns, beyond the same-arm spread at n=25 and n=29 and on the second Koblitz target. The first Koblitz target carries the worker start and CUDA initialisation and shows no difference within a 1.31 spread. The CPU `fes-f2` row moved 12.42 → 13.12 ms with a same-arm spread of 1.09 on this pod, so A/B 2 says nothing about it either way.

Where the call is now

At 30 unknowns, merged: 1.44 ms per call, of which the round trip is 1.42 ms and the kernel about 1.30 ms (A100 kernel round 2) [pcc:unverified]. The engine is now close to its kernel at the sizes where it is used for speed. Whole n=29 solve: 15.2 s (against `inherited-f4`'s 895.5 s in the earlier end-to-end run, on another pod); counted S is unchanged and `inherited-f4` remains cheaper in counted cost at every size measured.

Not done / next

- Batching several systems per launch: the descent oracle stops at the first liftable solution, so solving the next systems ahead of time would add counted work; only worth it with a different oracle contract.
- An independent replay before these rows are cited as more than `[pcc:unverified]`.